

PROPOSTA DE PROJETOS DE IMERSÃO

NOME DO PROJETO: Plataforma Guideway LMS – Escopo 1

EMPRESA PROPONENTE DO PROJETO: Willian Ferreira Winck

CONTATO NA EMPRESA: Willian Ferreira Winck

E-MAIL: willianfwinck@gmail.com

WHATSAPP: (51) 99891-0522

DESCRIÇÃO DO PROJETO PROPOSTO:

O objetivo do projeto **Guideway LMS** é construir uma plataforma de Learning Management System (LMS) robusta e completa, utilizando o CMS Joomla como base. A plataforma cobrirá todo o ciclo de vida de um curso online, desde a sua criação e gestão, passando pela entrega do conteúdo e interação entre alunos, até a emissão de certificados.

Para acelerar o desenvolvimento, o projeto será executado simultaneamente por dois grupos. O **Grupo 1** focará nas funcionalidades centrais da **Experiência do Aluno e Conteúdo**, desenvolvendo os recursos de aprendizado, avaliação e certificação. O Grupo 2 trabalhará em paralelo, focando nas ferramentas de comunicação e integração com IA.

TECNOLOGIAS E FERRAMENTAS:

- **Linguagens:** PHP 8.x, JavaScript (ES6+), HTML5, CSS3.
- **Banco de Dados:** MySQL / MariaDB.
- **CMS:** Joomla 5.
- **Extensões Principais (Stack Base):**
 - **LMS:** SP LMS (JoomShaper)
 - **Page Builder:** SP Page Builder (JoomShaper)

REQUISITOS FUNCIONAIS ESSENCIAIS:

1.1. Sistema Avançado de Certificados

- **Descrição:** O sistema atual de certificados é muito básico. Precisamos de um sistema robusto, personalizável e automatizado.
- **Tarefas:**
 - Criar um "Gerenciador de Certificados" na área administrativa, onde seja possível criar múltiplos templates.
 - Cada template deve permitir personalização avançada (imagem de fundo, posicionamento de texto, logo, assinatura, etc.).
 - Vincular um template de certificado a um ou mais cursos.

Apoio:

- **Automatizar a Geração:**
 - O certificado deve ser gerado dinamicamente com os dados do aluno (nome, nome do curso, data de conclusão).
 - Criar uma opção global para definir se a liberação do certificado é automática (ao concluir o curso) ou manual (requer aprovação do professor).
 - Desenvolver uma interface de moderação na área administrativa para que o professor possa visualizar os pedidos de certificado pendentes e aprová-los ou rejeitá-los.
 - Adicionar uma opção no curso para exigir a aprovação em todos os quizzes como critério para conclusão. Definir a nota mínima para aprovação (padrão: 70%).
- **Sugestão de Implementação Técnica:**
 - **Geração de PDF:** Utilizar uma biblioteca PHP como **TCPDF** ou **Dompdf** para gerar o arquivo PDF do certificado a partir de um template HTML/CSS.
 - **Templates:** Salvar os templates de certificado no banco de dados (o HTML/CSS e as coordenadas dos textos dinâmicos).
 - **Interface de Moderação:** Criar uma nova view no componente com_splms para listar os alunos aptos a receber o certificado, com botões de "Aprovar" e "Rejeitar".

1.2. Acompanhamento de Progresso

- **Descrição:** Atualmente, a extensão não possui nenhum tipo de controle para acompanhar o progresso do aluno. O objetivo é implementar um sistema do zero para rastrear quais aulas foram concluídas de forma precisa.
- **Tarefas:**
 - **Passo 1 (Backend): Estrutura de Dados e Endpoint**
 - **Banco de Dados:** Criar uma nova tabela no banco de dados (ex: #__splms_lesson_progress) para armazenar o status de cada aula para cada aluno. Colunas essenciais: id, user_id, lesson_id, status ('iniciado', 'concluído'), updated_at.
 - **Endpoint AJAX:** Criar um endpoint no componente com_splms que receberá uma requisição do frontend para marcar uma aula como "concluída". Este endpoint deve validar se o usuário está logado e inscrito no curso antes de salvar o progresso no banco.
 - **Passo 2 (Frontend): Implementação da API de Vídeo**
 - **Carregamento do Player:** Na página da aula no site, em vez de um <iframe> simples, usar JavaScript para carregar o player de vídeo do YouTube ou Vimeo dentro de um <div> específico, utilizando as APIs YouTube IFrame Player API ou Vimeo Player API.
 - **Monitoramento de Eventos:** Utilizar os "eventos" da API para detectar ações do usuário. O evento principal a ser monitorado é o de "vídeo concluído" (onStateChange para YT.PlayerState.ENDED no YouTube, ou on('ended') no Vimeo).
 - **Comunicação com Backend:** Quando o evento de "vídeo concluído" for disparado, o JavaScript deve enviar uma chamada AJAX para o endpoint criado no Passo 1, informando o ID da aula para que o progresso seja salvo.
 - **Passo 3 (Frontend): Interface do Usuário (UI)**
 - **Exibição do Progresso:** A lista de aulas do curso deve carregar já exibindo um indicador visual (ex: um ícone de "check") para as aulas que já foram marcadas como concluídas no banco de dados.
 - **Atualização em Tempo Real:** Após o vídeo ser concluído e o progresso ser salvo, o JavaScript deve atualizar a UI em tempo real (sem recarregar a página), mostrando o novo status de "concluído" para a aula recém-assistida.

Apoio:

1.3. Novo Tipo de Avaliação: Envio de Trabalho

- **Descrição:** Além de Quizzes, criar um tipo de avaliação onde o aluno precisa enviar um arquivo para ser corrigido pelo professor.
- **Tarefas:**
 - Adicionar um novo tipo de "Lição" chamado "Trabalho" ou "Projeto".
 - Nesta lição, o aluno poderá fazer o upload de um arquivo (PDF, ZIP, etc.).
 - O professor será notificado e terá uma interface para baixar o arquivo, corrigir e atribuir uma nota.
- **Sugestão de Implementação Técnica:**
 - **Upload de Arquivo:** Utilizar a classe JInputFiles do Joomla para processar o upload do arquivo de forma segura.
 - **Armazenamento:** Salvar os arquivos em uma pasta segura, fora da raiz pública do site, para evitar acesso direto. O nome do arquivo no servidor deve ser anonimizado para evitar conflitos (ex: usando md5(time() . \$userid)).
 - **Interface do Professor:** Criar uma nova view no admin para listar os envios por aluno e por curso.

REQUISITOS FUNCIONAIS DESEJÁVEIS:

1.4. Aulas ao Vivo

- **Descrição:** Permitir que professores vinculem aulas ao vivo em seus cursos.
- **Sugestão de Implementação Técnica:**
 - **Estrutura:** Criar um novo tipo de lição "Aula ao Vivo" (lesson_type = 'live').
 - **Campos no Admin:** Adicionar campos no formulário de edição da lição para: URL da Reunião, Data, Hora de Início, Duração Estimada.
 - **Lógica no Frontend:** Na página da aula, o layout deve exibir as informações da aula ao vivo. Pode-se usar JavaScript para, por exemplo, exibir um contador regressivo e só habilitar o botão de "Entrar na Aula" X minutos antes do horário de início.

1.5. Liberação Programada de Conteúdo (Drip Content)

- **Descrição:** Permitir que o professor programe a liberação das aulas, em vez de disponibilizar todo o conteúdo de uma vez.
- **Tarefas:**
 - Adicionar uma opção em cada módulo ou aula para definir quando ela será liberada (ex: X dias após a inscrição do aluno).
- **Sugestão de Implementação Técnica:**
 - **Banco de Dados:** Adicionar uma coluna na tabela de aulas para release_delay_days.
 - **Lógica de Acesso:** Na hora de exibir as aulas para um aluno, o código deve verificar a data de inscrição do aluno no curso e somar o release_delay_days. Se a data resultante for menor ou igual à data atual, a aula é exibida. Caso contrário, exibe uma mensagem "Disponível em X dias".

REQUISITOS NÃO FUNCIONAIS:

- **Usabilidade:** A interface para alunos e professores deve ser intuitiva e de fácil navegação.
- **Segurança:** O upload de arquivos e o tratamento de dados devem seguir as melhores práticas de segurança.

Apoio:

- **Desempenho:** As novas consultas ao banco de dados devem ser otimizadas para não impactar o carregamento das páginas.
- **Compatibilidade:** As funcionalidades devem ser compatíveis com os principais navegadores (Chrome, Firefox, Safari, Edge).

CRITÉRIOS DE ACEITAÇÃO:

- **Sistema de Certificados:**
 - **Dado** que um professor está na área administrativa, **ele deve ser capaz de** criar um novo template de certificado com uma imagem de fundo personalizada.
 - **Dado** que um aluno concluiu um curso com liberação automática, **ele deve ser capaz de** baixar seu certificado em PDF na área do aluno.
 - **Dado** que um curso requer aprovação do professor, **o professor deve ver** uma notificação ou uma lista de alunos aguardando aprovação.
- **Acompanhamento de Progresso:**
 - **Dado** que um aluno assistiu a um vídeo de uma aula até o final, **quando** ele voltar para a lista de aulas, **a aula assistida deve estar marcada** como "concluída".
 - **Dado** que o progresso está sendo salvo, **o sistema deve exibir** uma barra de progresso geral do curso (ex: "75% concluído").
- **Envio de Trabalho:**
 - **Dado** que uma aula é do tipo "Trabalho", **o aluno deve ver** um botão de "Enviar Arquivo".
 - **Dado** que um aluno enviou um trabalho, **o professor deve ser capaz de** acessar a página de envios, baixar o arquivo e inserir uma nota de 0 a 100.

Observação Importante: Existe um **Documento Base do Projeto** com informações gerais, regras, guias de estudo e configuração de ambiente que é de leitura obrigatória e se aplica a ambos os grupos.

CLAUSULA DE PROPRIEDADE INTELECTUAL

Os PARTÍCIPES (BRISA, UNISINOS, empresa e alunos) acordam, desde já, que todos os direitos sobre invenções, propriedade industrial e intelectual e direitos autorais oriundos das ações realizadas através deste ACORDO e/ou dos TERMOS ADITIVOS serão de propriedade da BRISA, da UNISINOS, dos alunos e das empresas parceiras dos projetos que serão desenvolvidos.

COMPROMISSO DE COLABORAÇÃO

Solicitamos à empresa que defina um profissional para tirar dúvidas dos alunos sobre as regras de negócio, escopo e tecnologia adotada. É muito importante que o profissional indicado participe da reunião de esclarecimento de escopo e das duas bancas de avaliações do projeto. As datas serão estabelecidas assim que o projeto iniciar.

Apoio: